Written Interview

Web Frontend Engineer – JS, CSS, React, Flutter

1. Web engineering experience

What skill or knowledge have you acquired in the past year that has been particularly helpful?

The most useful thing I have learned this year is how to use **AI properly as an engineering tool**, beyond simple autocomplete. I use Claude and GitHub Copilot every day and I work with agentic workflows. In these, the AI explores a codebase, proposes a plan, makes changes across several files and runs the tests, while I steer, review and decide. I have also learned how to design applications that use large language models: prompt and context design, tool use, structured outputs, evaluation, and the security and cost trade-offs. I am currently preparing for the Claude Certified Architect certification to make this knowledge structured and complete rather than something I picked up casually.

What motivated you to learn it?

I could see that AI was changing how software is built faster than any technology I have seen in my career. Many developers use it badly, either accepting its output blindly or ignoring it completely. I wanted to understand it well enough to get real value from it while keeping the quality bar high. As a Dev Lead I also felt responsible for this: if my team was going to use these tools, I needed to know where they help, where they fail, and how to review their output. Studying for a certification gave me a disciplined way to cover the parts I would not meet in daily work, such as evaluation and architecture of AI-based systems.

What has the impact been for you and your team?

For me, the biggest change is speed in the parts of the work that used to be slow but not difficult: understanding unfamiliar legacy code, writing test scaffolding, migrating repetitive patterns, and drafting documentation. That leaves more of my time for architecture, code review and the hard problems. It also helped my personal projects. As one engineer, I built and released a product on the web, Android and iOS, together with its back end, which would have taken much longer before.

For my team, I share what works and what doesn't: how to give the AI the right context, and why every AI-generated change goes through the same code review and tests as human code. We also agreed never to put client secrets or confidential data into prompts. The result is that we use AI to move faster without lowering quality, and the less experienced developers learn to treat it as a pair programmer whose work they must check, not as an oracle.

Describe your experience of web programming – JavaScript, TypeScript, React, CSS and Python in particular.

JavaScript (9 years). JavaScript has been my main language since 2017, from ES5/ES6 on the Western Union merchant portal up to ES2024 on Adobe Workfront today. I know the language in depth: closures, the event loop and asynchronous patterns (promises,

async/await, RxJS observables), modules, and tooling such as Webpack, Babel and ESLint. I also use JavaScript on the server with Node.js.

TypeScript. I use TypeScript daily. All my Angular projects (Fiserv, Cisco Webex, Hudson MX) were strictly typed, and so are my React Native and Node.js projects. I use types as documentation and as a safety net for API contracts. For example, I model payment states as discriminated unions so that impossible states cannot be represented, and I share request and response types between the front end and the Node.js back end so a contract change breaks the build instead of production.

React. At Adobe Workfront I lead front-end development on React, Redux, React Router and React Spectrum, Adobe's accessible design system, in a large shared codebase serving enterprise customers. At Mastercard I worked with React, GraphQL/Apollo and Ant Design on a payments management suite. There I untangled long-standing technical debt and delivered critical features on a dynamic form engine that reshapes the payment form for each client. I also have strong Angular experience (from Angular 5 to modern versions, with NgRx and RxJS), which gives me a useful comparison of how the two ecosystems handle state and change detection. In React Native I built a cross-platform crypto markets and portfolio app.

CSS. I write CSS by hand and care about it: SCSS, Flexbox, CSS Grid, Tailwind, responsive and component-based styling, and theming with design tokens. I have built UIs on several design systems (React Spectrum, Ant Design, PrimeNG, Bootstrap), so I understand them both as a consumer and from the side of keeping a component library consistent across teams. That is directly relevant to the Vanilla Framework. For me accessibility is part of good markup and CSS: semantic HTML, focus management, colour contrast and keyboard navigation.

Python. Python is my second back-end language after Node.js. I use it for small services and automation: a FastAPI service exposing REST endpoints, scripts that import, clean and transform data (for example CSV exports into a database), and tooling that calls LLM APIs as part of my AI work. I value Python for its readability and its excellent libraries, and I am comfortable building and maintaining back-end services in it alongside a team.

Describe your experience building large systems with many services – web front ends, REST APIs, data stores, event processing and other kinds of integration between components.

Most of my professional work has been building the front ends of large, multi-service systems, where the UI is the place in which many back-end services meet:

- **Adobe Workfront (Document Management)**: upload, preview, versioning and approval of creative assets inside enterprise workflows. These features depend on several services (storage, preview generation, versioning, permissions and workflow), and the UI must handle long-running asynchronous operations and partial failures gracefully.
- **Mastercard**: a payments management suite with a GraphQL/Apollo data layer over payment, account, user and user-group services, plus an admin layer for payment settings. The dynamic form engine was configuration-driven, so the front end had to stay correct for every client configuration.

- **Fiserv**: a payments application for creating, viewing and modifying internal and external payments over REST APIs, which I built from the first commit.
- **Western Union**: a merchant portal for small and medium enterprises with single and multiple points of sale, aggregating transaction and account data.

In my own products I own the whole stack. For my bill-splitting app, which is live on the web, Android and iOS, I designed and built the REST API in Node.js with TypeScript over a relational database. Some examples of the API design decisions:

- **Resource-oriented endpoints** such as `GET /groups/{id}/bills`, `POST /groups/{id}/bills` and `GET /groups/{id}/balances`, with consistent naming, HTTP verbs and status codes.
- **Idempotency keys** on POST requests, so that a mobile client retrying on a bad connection never creates the same bill twice. This is a lesson I brought from payments.
- **Cursor-based pagination** for bill history, and a single, consistent **error format** (code, message, field details) that every client can display properly.
- **Money stored as integer minor units** with an explicit currency, never as floating point, and balances calculated on the server so that the web and mobile clients always agree.
- **Versioned API** (`/v1`) and shared TypeScript types, so mobile apps already installed on users' phones keep working while the API evolves.
- **Authentication** with short-lived access tokens and refresh tokens, and authorisation checked on every request, so a user can only access groups they belong to.

For event-driven integration, my crypto app consumes real-time market data over WebSockets, merges it with REST history endpoints, and triggers push notifications when a price alert condition is met. On the server side, slow work such as sending notifications runs in background jobs rather than inside the request, so the API stays fast.

What are the key things to think about in regard to architecture, maintainability, and reliability in these large systems?

- **Clear contracts between components.** Well-designed REST APIs (or GraphQL schemas) are the real architecture of a multi-service system. They should be explicit, versioned, documented and typed end to end, so the front end and back end can evolve independently.
- **Clear ownership and boundaries.** Each service and front-end module should have one clear responsibility and owner. Shared component libraries such as a design system remove duplication without tying teams' release cycles together.
- **Designing for failure.** In a distributed system something is always failing somewhere. The UI needs explicit loading, empty, error and partial states. Back ends need timeouts, retries with backoff, and idempotent operations. In payments, idempotency is what prevents a retried request from becoming a double payment.
- **Data consistency and caching.** Decide deliberately where the source of truth is, how stale data may be, and how client caches are invalidated, especially for real-time data.
- **Asynchronous work and events.** Move slow or unreliable work into queues and background jobs, make consumers idempotent, and plan for events arriving late, twice or out of order.

- **Observability.** Logging, metrics, tracing and front-end error reporting, so the team finds issues before customers report them.
- **Automated testing at the right levels.** Unit tests for logic, component tests for UI behaviour, contract tests between services, and a small set of end-to-end tests for critical user journeys.
- **Safe delivery.** CI/CD, code review, feature flags and fast rollback. Small, frequent releases are less risky than big ones.
- **Simplicity and consistency.** Shared conventions, linting and documented decisions make a large codebase readable for the next engineer. I learned this when I set the patterns for the Fiserv application, which the rest of the team then followed.

Describe any experience you have with Flutter.

I use Flutter for my own products. My bill-splitting app is built with Flutter and Dart from a single codebase and is **live on the web, Android and iOS**. I was responsible for everything: the widget architecture, state management, integration with my REST API, responsive layouts that work from a phone to a desktop browser, and the release process to the App Store, Google Play and the web.

Building it showed me both Flutter's strengths and its trade-offs. The main strength is that one codebase and one rendering engine give the same, consistent UI on every platform, which is very efficient for a small team. The trade-offs show mostly on the web: initial bundle size, text selection and SEO behave differently from a normal DOM-based site, and accessibility needs deliberate attention through Flutter's semantics tree. Because I also build in React and React Native, I can judge when Flutter is the right tool and when a React web front end is the better choice. That is the kind of decision Canonical's teams make as they bring Flutter to desktop and web.

2. Software engineering experience

What kinds of software projects have you worked on before?

I have spent nine years at Endava delivering front-end solutions for global enterprise and fintech clients, and I build my own products alongside that:

- **Payment platforms** for Western Union (merchant portal), Fiserv (payments application built from scratch) and Mastercard (payments management suite with dynamic, per-client forms).
- **Insurance software** for Cisco Webex: complex, data-heavy interfaces for insurers modernising their operations.
- **Enterprise work management** at Adobe Workfront, where I am Dev Lead for Document Management.
- **Advertising technology** at Hudson MX: a cloud-based media-buying platform where I joined mid-project and became productive quickly in an unfamiliar codebase.
- **My own products**: an bill-splitting app in Flutter (web, Android, iOS) with a Node.js REST back end, and a React Native crypto markets and portfolio app.
- **Websites** built with WordPress and PHP.

Which operating systems, development environments, languages, databases and frameworks?

Area	Experience
Operating systems	macOS for daily development; Linux servers and containers for deployment; iOS and Android as target platforms
Development environments	WebStorm, Git/GitHub, CI/CD pipelines, Chrome DevTools, Xcode and Android Studio for mobile builds, Claude and Copilot
Languages	TypeScript, JavaScript (ES5–ES2024), HTML5, CSS3/SCSS, Dart, Python, PHP
Front-end frameworks	React, Angular, Flutter, React Native (Expo/EAS, Reanimated, Gesture Handler, React Navigation, Hermes), Redux / Redux Toolkit, NgRx, RxJS, GraphQL / Apollo Client, Transloco
Back end	Node.js, FastAPI (Python), REST API design, WebSockets
UI systems	React Spectrum, Ant Design, PrimeNG, Bootstrap, Tailwind CSS
Databases	PostgreSQL, MySQL (with WordPress), client-side storage on mobile and web
Testing	Jest, React Native Testing Library, Jasmine, Cypress (E2E), Flutter widget tests
Build tooling	Webpack, Babel, ESLint, EAS / TestFlight

Outline your thoughts on open-source software development. Have you been an open-source maintainer, and can you describe the scope of your contributions to those projects?

Almost everything I build stands on open source: React, Angular, Flutter, React Native, Node.js, Redux, RxJS, Webpack, Jest, Cypress and many more. I value the open-source model because it makes software better through transparency. Anyone can read the code, find a bug and propose the fix, and good ideas spread across the industry instead of staying locked inside one company. Open design systems are a good example. Spectrum's accessibility work, which I use every day, benefits everyone who studies or adopts it, and Vanilla can do the same for open-source projects that need a solid, accessible UI layer.

I also believe open source needs sustainable maintainership: clear governance, good documentation, a welcoming contribution process, and companies that pay engineers to maintain what they depend on. Canonical's model is a strong example of that.

I have not yet been an open-source maintainer. My professional work has been on private client codebases, and my own products are currently private repositories. Working in the open, as part of a company whose products are open source, is one of the main reasons I am applying to Canonical.

What software products have you yourself led which shipped many releases to multiple customers? What was your role?

- **Adobe Workfront – Document Management (2024–present), Dev Lead.** Workfront is used by many of the world's largest brands, and Document Management ships regular releases to all of them. I drive planning and delivery, unblock engineers, review code, and

keep developers, QA and design aligned. I am the first point of contact when a critical production issue needs diagnosing.

- **Fiserv payments application (2022–2023), front-end owner from the first commit.** I designed the architecture, set up the Angular workspace, implemented the core features and multi-language support, and set the patterns the team followed through its later releases.
- **My bill-splitting app (Flutter, Node.js), sole owner.** I designed, built and continue to release it on three platforms (web, Android and iOS) to real users. I own the product decisions, architecture, back end, releases and support.

What is your most senior role in a software engineering organisation?

My current role: **Senior UI Developer and Dev Lead** at Endava, leading the front-end team for Adobe Workfront's Document Management area. I joined Endava in 2017 as a UI Developer and was promoted from Mid-Level to Senior during the Cisco Webex engagement, in recognition of owning features from design handoff through end-to-end testing.

Describe your span of control and the diversity of products, functions and teams you led.

As Dev Lead I lead the front-end developers of the Document Management team and work every day with QA engineers, designers, product managers and back-end developers. I also coordinate with other Adobe teams that share the same codebase and design system. My role is technical and delivery leadership: I own front-end architecture decisions, plan and distribute work, set and maintain coding standards through code review, mentor developers, and represent the team when critical production issues need a fast, clear response.

Across my career I have worked in four product domains (payments, insurance, enterprise work management and advertising), with clients in different countries, time zones and cultures. This taught me to adapt quickly to a new team and codebase, which I showed on Hudson MX, where I joined mid-project and was shipping features within a short time. Outside client work, I have led full products myself from idea to release, including the product, design, back-end and mobile decisions.

One example of leadership I am proud of: on Workfront, users with very large folders reported that the document list became slow and sometimes froze when they selected or filtered assets. I took ownership of the investigation. Profiling with the React Profiler showed that each selection re-rendered the entire list, because a Redux selector returned a new array on every state change and some row components received new callback references each time. Together with one of the developers I was mentoring, we introduced memoised selectors, stabilised the callbacks, split each row into a memoised component, and virtualised the list so only the visible rows were rendered. Selection and filtering in large folders became instant, and the related customer complaints stopped. Just as important, we turned the lessons into code-review guidelines and a short session for the team, so the same problem does not return in other parts of the product.

Outline your views on the role of an engineering manager in shaping a high-functioning team.

I think a good engineering manager mostly creates the conditions in which good engineers can do their best work:

- **Clarity.** Everyone should know what we are building, why it matters to the user, and what "done" means. Most team friction comes from unclear goals, not from bad engineers.
- **Psychological safety.** People must feel safe to say "I don't understand", "I made a mistake" or "I think this design is wrong". I try to show this myself as a lead, for example by saying openly when I am not sure about something.
- **Removing blockers.** The manager's time is well spent unblocking others: getting a decision from product, clarifying an API with another team, or protecting focus time.
- **Growth.** Regular honest feedback, stretch tasks matched to each person, and mentoring. Teams stay strong when people can see themselves progressing.
- **High standards, shared by everyone.** Code review, testing and accessibility should be team habits, not personal preferences. The manager sets the bar and makes it easy to meet.
- **Healthy delivery rhythm.** Sustainable pace, small increments, and blameless retrospectives after incidents, so the team learns instead of hiding problems.
- **Hiring and team composition.** Hiring for ability, curiosity and communication, and building a team with complementary strengths.
- **A positive culture.** Energy and optimism are contagious, especially under pressure. I have always been the person who stays positive and supportive when things are difficult, and I have seen how much that helps a team keep focus and recover from setbacks.

Outline your thoughts on performance in software engineering. How do you ensure that your product is fast?

Performance is a feature that users feel even when they cannot name it. My approach is to **measure, set budgets, and keep it fast continuously** instead of fixing it once.

- **Measure first:** Chrome DevTools performance profiles, the React Profiler, Lighthouse and Core Web Vitals (LCP, INP, CLS), and on mobile, frame rates and cold-start time. I don't optimise based on assumptions.
- **Ship less code:** code splitting and lazy loading per route, tree-shaking, avoiding heavy dependencies, and watching bundle size in CI. This matters even more for Flutter on the web, where the initial download is a key cost.
- **Render less:** correct memoisation boundaries, stable references, normalised state, memoised selectors, and virtualised lists for large data sets. The Workfront document-list example above is a typical case. In React Native, the same techniques plus Hermes and Reanimated are what made 60fps interactions possible.
- **Fetch smarter:** pagination, caching and request de-duplication, avoiding request waterfalls, and good API design. A well-shaped REST endpoint that returns exactly what the screen needs is often the biggest performance improvement.
- **Fast back ends:** proper database indexes, avoiding N+1 queries, and moving slow work into background jobs.

- **Perceived performance:** skeletons, optimistic updates and progressive rendering, so the interface responds immediately.
- **CSS and assets:** modern layout instead of JavaScript-driven layout, optimised images and fonts, and avoiding layout thrashing.
- **Guard against regressions:** performance budgets and checks in CI, so the product stays fast as the team adds features.

Outline your thoughts on quality in software development. What practices are most effective in software teams to drive improvements in quality?

Quality is not a phase at the end. It is the result of many small habits shared by the whole team. The practices I have seen work best:

- **Thoughtful code review** focused on correctness, readability and design, not just style. Linters and formatters should handle style automatically so people can focus on what matters.
- **Strong typing:** TypeScript and Dart catch a whole class of bugs before they exist, especially at API boundaries.
- **A balanced test suite:** fast unit and component tests, plus end-to-end tests for critical journeys. On Cisco Webex I covered critical user flows with Cypress, which gave the team confidence to refactor.
- **CI that blocks merges** on failing tests, lint or type errors, and small pull requests that are easy to review.
- **Design systems:** reusable, accessible, well-tested components mean each bug is fixed once, for every product.
- **Accessibility as part of the definition of done:** semantic HTML, keyboard navigation, screen-reader checks and contrast are quality issues, not extras.
- **Close collaboration with QA and design** from the start, so requirements and edge cases are clear before coding begins. In payments, edge cases are where the real bugs live.
- **Responsible use of AI tools:** AI-generated code is reviewed and tested exactly like human code. AI can also raise quality by generating test cases and spotting edge cases a reviewer might miss.
- **Learning from incidents:** blameless post-mortems, and turning every production bug into a regression test.
- **Paying down technical debt deliberately**, as I did at Mastercard, instead of letting it accumulate until it blocks delivery.

3. Education

How did you rank in your final year of high school in mathematics? Were you a top student? On what basis would you say that?

I studied at the Theoretical Lyceum "Mihai Viteazul" and was the top student in my class overall in my final year. In mathematics I was among the strong students of my class. I scored **9 out of 10 in the national Baccalaureate (BAC) examination in**

mathematics, which is a high result on a national, externally marked exam. I say this on the basis of the national exam result and my school grades. To be honest, mathematics was a strong subject for me but not my very strongest one; my best results were in Romanian, biology and English.

How did you rank in your final year of high school, in your home language? Were you a top student? On what basis would you say that?

Yes. Romanian was my language of instruction, and I achieved the **maximum grade of 10 out of 10 in the national Baccalaureate examination in Romanian language and literature**. Together with my overall position as top student in my class, this is the basis for saying I was a top student in my home language. I have always loved reading, which helped me a lot in literature and writing.

Please state your high school graduation results or university entrance results, and explain the grading system used.

In Moldova, high school ends with the national Baccalaureate examination (BAC). Exams are written in the same conditions across the country and are marked externally. Results use a 1–10 scale, where 10 is the highest and 5 is the minimum pass. University admission is based mainly on the BAC results.

- Romanian language and literature: **10 / 10** (the maximum)
- Mathematics: **9 / 10**
- Overall: top student in my class in the final year

Can you make a case that you are in the top 5% in your academic year, or top 1%, or even higher? If so please outline that case.

Yes, I believe I can make a well-supported case for the top 5% of my academic year, both at school and at university:

- **High school:** top student in my class; the maximum mark (10/10) in the national BAC exam in Romanian and 9/10 in mathematics; and selection to represent my school at the **regional olympiads in Biology and English**. Only the best students from each school are chosen for olympiads.
- **University:** a final average of **9.7 out of 10**, a merit scholarship, and the **Gold Medal as the best student of the year**, which goes to the single strongest student. That last award is the clearest evidence that I was at the very top of my year.
- **International selection:** I was chosen through a competitive selection for an **11-month Erasmus Mundus exchange at the Czech Technical University in Prague**. Places are limited and awarded on academic merit.
- **Choice of universities:** I was also offered a place with a scholarship at the Technical University of Cluj-Napoca in Romania, one of the leading technical universities in the region.

Being named best student of the year suggests a position well within the top 1% at university, while my school results support the top 5%.

What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for?

I was a hard-working, curious and consistent student who enjoyed learning across very different subjects, from biology to languages to mathematics. Outside class my great passion was reading books. Reading widely shaped how I think and write, and it is still one of my favourite ways to learn. I also spent time preparing for the biology and English olympiads, which taught me to study deeply and independently, beyond the curriculum.

I think my classmates would remember me for being **very positive in any situation and very supportive**. I was the person others came to when they needed help with a subject, encouragement before an exam, or simply someone to listen. I still bring this to my teams today.

Which university and degree did you choose? What other universities did you consider, and why did you select that one?

I chose a **Bachelor's degree in Information Technology at the Technical University of Moldova (2012–2016)**, the country's leading technical university and the main source of engineers for Moldova's IT industry.

I also seriously considered the **Technical University of Cluj-Napoca** in Romania, one of the strongest technical universities in the region, which offered me a place with a scholarship. I declined it because, at that time, a personal family matter meant it was important for me to stay close to my family. It was a difficult decision, but the right one. I committed fully to my studies in Chișinău and graduated as the best student of my year. Later, through the Erasmus Mundus programme, I still gained a long period of international study.

Overall, what was your degree result and how did that reflect on your ability? Please help us understand the grading system for your results.

Moldovan universities grade on a 1–10 scale, where 10 is the highest and 5 is the minimum pass. Averages above 9 are considered excellent and are reached by a small minority of students. I graduated with a **final average of 9.7 out of 10**, studied on a **merit scholarship**, and received the **Gold Medal as the best student of the year**.

I think this shows not only ability but consistency. A 9.7 average means performing at the highest level across four years and across very different subjects: mathematics, programming, networks, databases and theory. That consistency is how I still work today.

During my studies I also spent **11 months at the Czech Technical University in Prague** through the Erasmus Mundus programme. Studying in English, in a different academic system and an international environment, taught me to adapt quickly and to communicate with people from many cultures. That has been very valuable in my career with international clients.

During all of your education years, from high school to university, can you describe any achievements that were truly exceptional?

- **Gold Medal as the best student of the year** at the Technical University of Moldova.
- **Final university average of 9.7/10**, with a merit scholarship.

- **Maximum grade (10/10) in the national Baccalaureate exam in Romanian**, and 9/10 in mathematics.
- **Regional olympiad participant in Biology and English**, selected to represent my school.
- **Competitive selection for an 11-month Erasmus Mundus exchange** at the Czech Technical University in Prague.
- **Scholarship offer from the Technical University of Cluj-Napoca**.

What leadership roles did you take on during your education? Did you conceive of, and drive to completion, any initiatives outside of your required classwork?

I did not hold formal titles such as class president. My leadership during my education was mostly informal, and it came from the same qualities my classmates remember me for. I regularly helped classmates prepare for exams and explained difficult topics, and I was often the one who kept a group motivated and positive under pressure.

Outside required classwork, my biggest initiatives were self-driven. I prepared for the biology and English olympiads beyond the school curriculum. I applied for and completed an 11-month study period abroad in Prague, which required independence and adaptation in a new country and language. Learning that I could take full responsibility for my own goals, and help others reach theirs, prepared me for the Dev Lead role I hold today.

4. Context

Outline your thoughts on the mission of Canonical. What is it about the company's purpose and goals which is most appealing to you? What is risky or unappealing? Are there any elements of the company goals that you are unsure about?

As I understand it, Canonical's mission is to make open-source software secure, reliable and easy to use for everyone, from a single developer's laptop to the largest clouds, data centres and edge devices, and to fund that work sustainably through enterprise support and services. Ubuntu is the visible part of the mission. Behind it are long-term security maintenance, cloud and Kubernetes tooling, IoT, and the web products that let people discover, install and manage all of this.

What appeals to me most is that the work is open and has very wide reach. Improvements made here are used by millions of people and organisations, not only one client. As a front-end engineer I find the Vanilla Framework especially meaningful: a shared, accessible presentation layer that brings consistency to Canonical's products and can be adopted by other open-source projects that often have no designers or front-end specialists. Accessibility done well in a shared toolkit spreads to every product built on it. I also like that Canonical treats design as a partner of engineering, and that it is investing in Flutter, a technology I already use and believe in.

What I see as risky: Canonical competes against companies with much larger budgets while also giving much of its work away for free. That requires very careful focus. Some product decisions, such as Snap, have caused friction with parts of the community, and

being both a community steward and a commercial vendor is a difficult balance. The globally distributed, remote-first model with a demanding bar is a great strength, but it needs excellent written communication and self-discipline. I see that as a challenge I would enjoy rather than a negative.

What I am unsure about: the long-term role of Flutter on the web. From my own experience, Flutter is excellent for unifying mobile and desktop, but on the web it still has trade-offs in initial load size, SEO and accessibility compared with React. It also depends on Google's continued investment. I would like to understand how Canonical decides which products should be built in Flutter and which should stay in React.

Who are competitors to Canonical, and what does Canonical need to change to be a more effective competitor?

- **Enterprise Linux:** Red Hat (IBM) with RHEL and OpenShift, SUSE, and community rebuilds such as Rocky Linux and AlmaLinux.
- **Cloud providers' own platforms:** Amazon Linux, Azure Linux, Google's Container-Optimized OS, and managed Kubernetes services that reduce the need for a separately supported operating system.
- **Developer desktop:** macOS (which I use myself) and Windows with WSL, which, ironically, often runs Ubuntu.
- **Private cloud and infrastructure:** VMware (Broadcom) and Nutanix, and Red Hat OpenShift for Kubernetes.
- **Secure container images and the software supply chain:** companies such as Chainguard, and Docker's hardened images.

To compete more effectively, I think Canonical should:

- Make the **web experience** of its products as polished and self-service as the best SaaS products: clear docs, fast and accessible dashboards, and simple trial and purchase flows. Enterprise buyers often judge a platform by its interfaces first, and this is where web engineering has direct commercial impact.
- Communicate its **security and long-term support value** more clearly, especially now, when Red Hat's source policy and VMware's licensing changes are pushing customers to look for alternatives.
- Win over more **developers on macOS and Windows**, for example through excellent WSL, container and virtual-machine experiences, so Ubuntu stays the default wherever developers work.
- Keep strengthening **community trust** through openness about decisions such as Snap, so developers remain advocates.
- Grow the **Vanilla Framework** outside Canonical, with strong documentation, React components and examples, so it becomes a recognised open-source design system and a channel to developers.

What would you most want to change about Canonical?

From the outside, I would most like Canonical's design and front-end work to become more visible to the wider developer community. Vanilla is a strong project, but it is much less known than Material UI, Carbon or Spectrum. More public documentation, examples, talks

and contributions to accessibility standards would attract more open-source projects to it, and would help hiring too. I would be glad to contribute to that. I am aware this is an outside view, and I would first want to understand the reasons behind current priorities before proposing changes.

What gets you most excited about this role?

- **Working in the open.** After nine years on private client codebases, the chance to build software that anyone can use, read and improve is the main reason I am applying.
- **Design systems and accessibility at scale.** I have worked on top of React Spectrum and other design systems for years. Contributing to Vanilla and helping more projects build accessible, responsive interfaces is exactly the work I enjoy most.
- **React and Flutter together.** I already build with both, and I would love to help Canonical use each where it fits best and bring Flutter to desktop and web.
- **Full-stack ownership and well-designed APIs.** I enjoy designing REST APIs and building back-end services in Node.js and Python, and this role allows front-end engineers to do both.
- **AI-assisted engineering.** I would like to bring my experience with AI tools and agentic workflows to a team that cares about quality as much as I do.
- **The people and the way of working.** A global, remote-first, high-bar team of engineers and designers that I can learn from, and to which I can bring my experience of leading, mentoring and keeping a team positive under pressure.